AI Assisted Coding

Assignment 12.3

Name: Vivek

Hallticket:2303A52009

Task 1: Sorting Student Records for Placement Drive
Scenario
SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
Tasks
1. Use GitHub Copilot to generate a program that stores student records
(Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
o Quick Sort
o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.
Expected Outcome
• Correctly sorted student records.
• Performance comparison between Quick Sort and Merge Sort.
• Clear output of top-performing students.

Prompt:
""" assume you are a developer for a university where Placement cell needs to shortlist the students efficiency during campus placement exam must be sorted by CGPA in decending order
you must use the quick sort and merge sort to sort the CGPA of students
measure and compare runtime
write a display function to show the top 10 students based on CGPA descending order """

CODE:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x > pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x < pivot]
    return quick_sort(left) + middle + quick_sort(right)

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] > right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

def display_top_students(students, n=10):
    print(f"Top {n} students based on CGPA:")
    for i in range(min(n, len(students))):
        print(f"{i+1}. {students[i][0]} - CGPA: {students[i][1]}")
# Generate random student data
students = [(f"Student_{i}", round(random.uniform(2.0, 4.0), 2)) for i in range(100)]
# Sort using quick sort
start_time = time.time()
sorted_students_quick = quick_sort(students)
end_time = time.time()
print(f"Quick Sort Time: {end_time - start_time:.6f} seconds")
# Sort using merge sort
start_time = time.time()
sorted_students_merge = merge_sort(students)
end_time = time.time()
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes> python -u "c:\Users\
Quick Sort Time: 0.000206 seconds
Merge Sort Time: 0.000303 seconds
Top 10 students based on CGPA:
1. Student_99 - CGPA: 3.66
2. Student_98 - CGPA: 2.26
3. Student_97 - CGPA: 2.69
4. Student_96 - CGPA: 2.25
5. Student_95 - CGPA: 2.01
6. Student_94 - CGPA: 3.85
7. Student_93 - CGPA: 2.53
8. Student_92 - CGPA: 3.28
9. Student_91 - CGPA: 3.87
10. Student_90 - CGPA: 3.43
Top 10 students based on CGPA:
1. Student_99 - CGPA: 3.66
2. Student_98 - CGPA: 2.26
3. Student_97 - CGPA: 2.69
4. Student_96 - CGPA: 2.25
5. Student_95 - CGPA: 2.01
6. Student_94 - CGPA: 3.85
8. Student_92 - CGPA: 3.28
9. Student_91 - CGPA: 3.87
10. Student_90 - CGPA: 3.43
```

Task_02:

Task 2: Implementing Bubble Sort with AI Comments

- **Task: Write a Python implementation of Bubble Sort.**
- **Instructions:**
- **Students implement Bubble Sort normally.**
- **Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).**
- **Request AI to provide time complexity analysis.**
- **Expected Output:**
- **A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.**

Prompt:

```python
""" give me the code for  Bubble sort with inline comments and explain the code in detail """
def bubble_sort(arr):
    n = len(arr)  # Get the length of the array
    # Traverse through all elements in the array
    for i in range(n):
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]  # Swap the elements
    return arr  # Return the sorted array
# Example usage
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]  # Sample array to be sorted
    print("Original array:", arr)  # Print the original array
    sorted_arr = bubble_sort(arr)  # Sort the array using bubble sort
    print("Sorted array:", sorted_arr)  # Print the sorted array
# Explanation of the code:
# The bubble_sort function takes an array as input and sorts it in ascending order using the bubble
# sort algorithm. The outer loop runs n times, where n is the length of the array. The inner loop compares
# adjacent elements and swaps them if they are in the wrong order. After each pass through the array, the largest
# element is "bubbled" to the end of the array, which is why we can ignore the last i elements in subsequent passes.
```

Output:

```
TERMINAL
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes\lab_12.3.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes>
```

Task 3: Quick Sort and Merge Sort Comparison
• Task: Implement Quick Sort and Merge Sort using recursion.
• Instructions:
• Provide AI with partially completed functions for recursion.
• Ask AI to complete the missing logic and add docstrings.
• Compare both algorithms on random, sorted, and reverse-sorted lists.
• Expected Output:
• Working Quick Sort and Merge Sort implementations.
• AI-generated explanation of average, best, and worst-case complexities
Prompt:
""" compare the both quick & merge sort using recursion • Compare both algorithms on random, sorted, and reverse-sorted lists. AI-generated explanation of average, best, and worst-case complexities"""

```python
""" compare the both quick & merge sort using recursion  • Compare both algorithms on random, sorted, and reverse-sorted lists. AI-generated explanation of aver
import random
import time
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    merged = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
```

Output:

```
Merge Sort on sorted list: 0.001075 seconds
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes> python -u "c:\Users\Vivek\OneD
Quick Sort on random list: 0.000951 seconds
Merge Sort on random list: 0.002821 seconds
Quick Sort on sorted list: 0.001006 seconds
Merge Sort on sorted list: 0.002849 seconds
Quick Sort on reverse-sorted list: 0.001044 seconds
Merge Sort on reverse-sorted list: 0.002573 seconds
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes>
```

Task 4 (Real-Time Application – Inventory Management System)
Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff
need to:
1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.
Task:
• Use AI to suggest the most efficient search and sort algorithms for this use case.
• Implement the recommended algorithms in Python.
• Justify the choice based on dataset size, update frequency, and performance requirements.
Expected Output:
• A table mapping operation → recommended algorithm → justification.
• Working Python functions for searching and sorting the inventory.

CODE:

```python
class Product:
    def __init__(self, pid, name, price, quantity):
        self.pid = pid
        self.name = name
        self.price = price
        self.quantity = quantity
    def __repr__(self):
        return f"{self.pid} | {self.name} | Price:{self.price} | Qty:{self.quantity}"
# Binary Search (Search by Product ID)
def binary_search(products, target_id):
    low = 0
    high = len(products) - 1
    while low <= high:
        mid = (low + high) // 2
        if products[mid].pid == target_id:
            return products[mid]
        elif products[mid].pid < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Linear Search (Search by Name)
def linear_search(products, name):
    for product in products:
        if product.name.lower() == name.lower():
            return product
    return None
# Quick Sort by Price
```

Output:

```
Search by Product ID:
103 | Keyboard | Price:1200 | Qty:80

Search by Product Name:
104 | Monitor | Price:12000 | Qty:40

Products Sorted by Price:
102 | Mouse | Price:500 | Qty:150
103 | Keyboard | Price:1200 | Qty:80
105 | Printer | Price:8000 | Qty:20
104 | Monitor | Price:12000 | Qty:40
101 | Laptop | Price:75000 | Qty:15

Products Sorted by Quantity:
101 | Laptop | Price:75000 | Qty:15
105 | Printer | Price:8000 | Qty:20
104 | Monitor | Price:12000 | Qty:40
103 | Keyboard | Price:1200 | Qty:80
102 | Mouse | Price:500 | Qty:150
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes>
```

Task_05:
Task 5: Real-Time Stock Data Sorting & Searching
Scenario:
An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.
• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
• Implement sorting algorithms to rank stocks by percentage change.
• Implement a search function that retrieves stock data instantly when a stock symbol is entered.
• Optimize sorting with Heap Sort and searching with Hash Maps.
• Compare performance with standard library functions (sorted(), dictlookups) and analyze trade-offs.

Prompt:
""" Generate a short solution for a Python assignment on **Real-Time Stock Data Sorting & Searching.**

Scenario:
A FinTech lab tool analyzes stock price movements. Each stock contains:

*** Stock Symbol**
*** Opening Price**
*** Closing Price**

Tasks:

1. Simulate a small dataset of stock prices.
2. Calculate percentage change for each stock.
3. Implement **Heap Sort to rank stocks by percentage gain/loss.**
4. Implement **Hash Map (dictionary) search to quickly retrieve stock data by stock symbol.**
5. Compare the performance with Python built-in methods **sorted() and **dictionary lookups**.**

6. Provide:

*** Short explanation of the Python functions for sorting and searching**

*** Example outputs.**

"""

Code:

```python
import random
# Simulate stock data
stocks = [
    {"symbol": "AAPL", "open": 150, "close": 155},
    {"symbol": "GOOGL", "open": 2800, "close": 2750},
    {"symbol": "AMZN", "open": 3400, "close": 3450},
    {"symbol": "MSFT", "open": 300, "close": 310},
    {"symbol": "TSLA", "open": 700, "close": 680},
]
# Calculate percentage change
for stock in stocks:
    stock["change"] = ((stock["close"] - stock["open"]) / stock["open"]) * 100
# Heap Sort implementation to rank stocks by percentage gain/loss
def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and arr[l]["change"] > arr[largest]["change"]:
        largest = l
    if r < n and arr[r]["change"] > arr[largest]["change"]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
# Sort stocks by percentage change
heap_sort(stocks)
# Create a hash map (dictionary) for quick retrieval by stock symbol
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\Codes> python -u "c:\User
Stocks sorted by percentage change (Heap Sort):
TSLA: -2.86%
GOOGL: -1.79%
AMZN: 1.47%
MSFT: 3.33%
AAPL: 3.33%

Stocks sorted by percentage change (Built-in sorted()):
MSFT: 3.33%
AMZN: 1.47%
MSFT: 3.33%
AAPL: 3.33%

Stocks sorted by percentage change (Built-in sorted()):
MSFT: 3.33%
Stocks sorted by percentage change (Built-in sorted()):
MSFT: 3.33%
AAPL: 3.33%
AMZN: 1.47%
AAPL: 3.33%
AMZN: 1.47%
AMZN: 1.47%
GOOGL: -1.79%
TSLA: -2.86%
```